

Tema 2: Comunicación

Socket: es una de las dos partes en una comunicación de red que interactúan para enviar o recibir datos. Es el lugar donde se origina o termina una conexión. Definido por:

- Dirección IP: ID único del nodo en la red.
- Número de puerto: especifica una aplicación o servicio en ese nodo.

Es un conjunto reducido de llamadas al sistema, que cubren cuatro tareas:

- Crear el conector
- Darle una dirección
- Establecer la conexión
- Transferir los datos

Un conector (socket) trabaja siempre con bytes, no con el tipo de dato que maneja el programa.

Datagramas (UDP, sin conexión) vs Conexión (TCP, transporte fiable):

-Datagramas: Cada mensaje va por separado, con su destino. Ligero y sin establecimiento previo, pero sin garantía de entrega ni orden. **Se usa donde importe la latencia y una muestra pérdida no sea grave**: telemetría periódica, descubrimiento en la red local, difusión a varios nodos.

-Conexión: Las dos partes establecen primero un canal y luego intercambian un flujo fiable y ordenado. Más sobrecarga, pero nadie gestiona pérdidas. **Se usa donde cada byte cuente**: una orden a un robot, una transferencia, una petición-respuesta.

Funciones para programar sockets (primitivas):

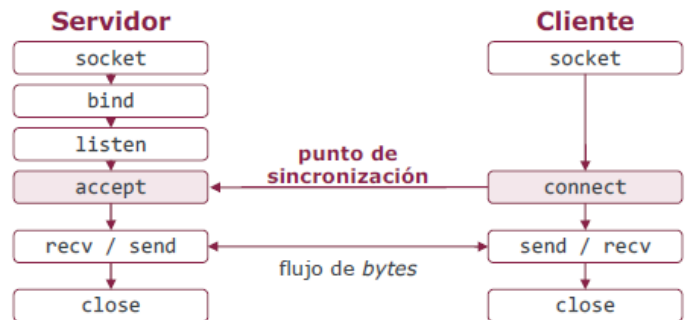
Estas diez operaciones son toda la interfaz: unas preparan el conector, otras establecen la conexión y las últimas transfieren los datos y la cierran.

accept y *recv* bloquean al proceso que las llama, y ese bloqueo es lo que obliga a atender a cada cliente en su propio hilo.

Operación	Descripción
<code>socket()</code>	Crea un nuevo punto final de comunicación
<code>bind()</code>	Adjunta una dirección local a un conector
<code>listen()</code>	Marca el conector como pasivo y sugiere al sistema el tamaño de la cola de conexiones pendientes
<code>accept()</code>	Toma la siguiente conexión de la cola y bloquea al llamante si no hay ninguna
<code>connect()</code>	Intenta activamente establecer una conexión
<code>send()/recv()</code>	Envía y recibe datos por una conexión TCP
<code>sendall()</code>	Reintenta hasta enviar todos los bytes. Si falla, levanta excepción y no informa de cuántos salieron
<code>sendto()/recvfrom()</code>	Envía y recibe datagramas UDP, sin conexión
<code>close()</code>	Libera la conexión

Secuencia de primitivas:

- El servidor se prepara antes de escuchar: reserva su dirección con **bind** y abre la cola de conexiones pendientes con **listen**.
- **accept** es el punto de sincronización. El servidor se queda ahí bloqueado hasta que un cliente hace **connect**.
- **accept** devuelve un conector nuevo, dedicado a esa conversación, y el original sigue escuchando a los demás.
- Cada extremo alterna **send** y **recv** sobre ese conector, y **close** libera los dos extremos.
- Si el servidor no reserva la dirección, el sistema le asigna un puerto cualquiera y el cliente no sabe adónde llamar.



Tipos de conectores y transporte:

El par que se abre, tipo de conector y protocolo, fija el transporte que se usa por debajo:

- SOCK_STREAM sobre TCP
- SOCK_DGRAM sobre UDP

Así se elige en Python

`s = socket(AF_INET, SOCK_STREAM) → TCP` `s = socket(AF_INET, SOCK_DGRAM) → UDP`
El primer argumento es la familia de direcciones: AF_INET para IPv4 y AF_INET6 para IPv6.

TCP entrega un flujo de bytes ordenado y fiable, pero sin fronteras: nada garantiza que cada **recv** corresponda a un **send**.

Dos mensajes pueden llegar pegados en una sola lectura, o uno partirse en dos.

La aplicación tiene que imponer su propia delimitación (framing):

- Prefijo de longitud. Se antepone el tamaño del mensaje y el receptor lee exactamente esos bytes.
- Delimitador acordado. Se cierra el mensaje con una marca que no aparezca en los datos.

Con datagramas el problema no existe: UDP conserva la frontera de cada mensaje, pero puede perderlo o entregarlo desordenado.

El servidor envía `send("HOLA")` `send("MUNDO")`
y el cliente puede leer `recv() -> "HOLAMUN"` `recv() -> "DO"`

Ejemplo en Python cliente-servidor:

-Clase servidor:

- **socket** crea el extremo, **bind** lo asocia al puerto y **listen** fija la cola de espera.
- **accept** bloquea hasta que llega un cliente y devuelve un conector nuevo, dedicado a esa conversación, y la dirección del cliente.
- **recv** bloquea hasta que hay datos y devuelve como mucho 1024 bytes: el conector trabaja con bytes, no con mensajes.
- Un **recv** vacío significa que el cliente cerró, y es la condición de salida del bucle.
- Este servidor atiende a un cliente a la vez. Para varios hace falta un hilo o un proceso por conexión.

```
01 from socket import *
02
03 class Server:
04     def run(self):
05         s = socket(AF_INET, SOCK_STREAM)
06         s.bind((HOST, PORT))
07         s.listen(1)
08         (conn, addr) = s.accept()
09
10         while True:
11             data = conn.recv(1024)
12             if not data: break
13             conn.send(data + b"")
14
15         conn.close()
16
```

-Clase cliente:

- **connect** bloquea hasta que el servidor acepta la conexión.
- **send** y **recv** trabajan sobre un flujo de bytes: nada garantiza que una llamada a **send** se corresponda con una a **recv** en el otro extremo.
- Aquí el final lo marca el cierre: un **send** de cero bytes no transmite nada, y lo que desbloquea el **recv** del servidor devolviendo vacío es que el otro extremo cierre. Si el diálogo tuviera que seguir, el fin de cada mensaje habría que acordarlo con un delimitador o un prefijo de longitud: es protocolo de aplicación y el conector no lo trae puesto.
- **close** libera el extremo. Sin él, el servidor se queda esperando en su **recv**.

```
01 from socket import *
02
03 class Client:
04     def run(self):
05         s = socket(AF_INET, SOCK_STREAM)
06         s.connect((HOST, PORT))
07
08         s.send(b"Hello, world")
09         data = s.recv(1024)
10         print(data)
11
12         s.send(b"")
13         s.close()
14
```

Representación externa de los datos:

Emisor y receptor necesitan formato común:

- Ordenación de los (bytes endianness): Unas ponen primero el byte más significativo. Otras lo ponen al revés.
- Codificación del texto (UTF-8, Latin-1...): Los mismos bytes son texto distinto según la tabla usada.
- Formato de la coma flotante (IEEE 754): El formato de los reales no es igual en todas las arquitecturas.

La solución es acordar una representación externa de datos: un formato común e independiente de la máquina, al que se traduce antes de enviar y desde el que se traduce al recibir.

- Serializar (marshalling) es pasar de la representación interna al formato común.
- Deserializar (unmarshalling) es la operación inversa en el receptor.

Evolución del esquema:

Emisores y receptores se actualizan en momentos distintos, así que el esquema tiene que poder cambiar sin romper a ninguno. El número de campo es la clave con la que viaja cada dato, y una vez publicado no se toca.

- Añadir un campo con un número nuevo es seguro en las dos direcciones. Un lector antiguo no conoce esa clave y la ignora. Un lector nuevo que recibe un mensaje sin ella obtiene el valor por omisión del tipo.
- Quitar un campo obliga a reservar su número con reserved, para que nadie lo reutilice y reviva datos antiguos con otro significado.
- Cambiar el tipo de un campo publicado no se puede, salvo entre tipos que comparten codificación.

Esto es lo que da un esquema y no da un JSON suelto: los dos extremos se despliegan por separado, en el orden que sea, sin coordinarse

Middleware de comunicación